SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5116

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Assessment Tool Weightage**:** 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Debugging & Optimization Task

Code of Conduct:

I (Deepak Kumar B/192424227) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A Debugging & Optimisation Task

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)

A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.
B. Optimize the code to reduce execution time for large input datasets (≥10 MB).
C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

2. Performance Analysis of Block Cipher Modes of Operation A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

A. Debug incorrect padding and IV handling causing decryption failures.
B. Refactor the code to minimize redundant computations and memory usage.
C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

3. Debugging RSA Cryptosystem Implementation A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

A. Identify issues in prime generation, key generation, and modular exponentiation.
B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
C. Evaluate the trade-offs between key size, security, and computational performance.

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

A. Debug incorrect shared key generation due to improper modular arithmetic.
B. Optimize exponentiation operations for large prime values.
C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

A. Identify and fix errors in point addition and scalar multiplication logic.
B. Optimize the implementation using efficient algorithms (e.g., double-and-add).

Debugging an RSA Cryptosystem Implementation

Question 3: Problem identification, debugging, optimization and trade-off analysis

Scenario

A Python RSA encryption system fails when encrypting large messages and occasionally returns incorrect plaintext after decryption. Two symptoms point to two different fault classes. A failure that depends on message size points to the message-to-integer step. An error that appears only now and then points to a hidden flaw in the keys (a composite number accepted as prime) or to data-dependent corruption (lost leading zeros, floating-point rounding).

RSA correctness rests on one identity: for a valid key pair and any 0 <= m < n, (m^e)^d mod n = m. Every defect below breaks one of the conditions that make this identity hold.

Part A: Problem Identification

The defects are grouped by the three areas named in the question, plus message handling, which explains the large-message failure.

Component	Defect	Root cause	Symptom
Prime generation	Primality tested by trial division or a Fermat test, or with too few Miller-Rabin rounds	Composite numbers (e.g. Carmichael numbers such as 561) pass as primes, so phi(n) is computed wrongly	Round trip occasionally returns wrong plaintext; key looks valid but is not
Prime generation	Uses random instead of secrets; top bit and low bit not forced	Predictable PRNG; n can be shorter than intended; even candidates	Weak or guessable keys; unexpected key length
Prime generation	p equal to q, or p and q very close	No check after generating both primes	$n = p^2$ is trivially factored; Fermat factorisation breaks close primes
Key generation	gcd(e, phi) not checked	e must be coprime to phi(n) for an inverse to exist	Inverse fails or d is wrong, so decryption is wrong
Key generation	d found by a brute-force loop, or a hand-written inverse that returns a negative value	Slow search; negative result of extended Euclid not reduced mod phi	Hangs on large keys; wrong d for some key pairs
Key generation	Tiny e (such as 3) with no padding; very small d	Cube-root and Wiener attacks apply	Insecure keys even when the code works
Modular exponentiation	(m ** e) % n computes the full power first	Intermediate result has about $e \times \log_2(m)$ bits	Extreme slowness or memory errors on large inputs
Modular exponentiation	Floating-point arithmetic (** with floats, math.pow)	Floats keep only about 53 bits of precision	Silent rounding, so occasional wrong plaintext
Message	Message integer m >= n;	RSA works only for 0 <= m <	Large messages fail:

handling	whole message converted to one integer	n, so the result wraps modulo n	decryption returns m mod n
Message handling	Bytes-to-integer conversion drops leading zero bytes	int.from_bytes loses the length of the original data	Output is shorter than the input, so wrong plaintext for some messages
Message handling	No padding (textbook RSA)	Deterministic and malleable	Pattern leakage and chosen-ciphertext attacks

Why the failures are size-dependent and intermittent

- **Large messages:** RSA operates on integers smaller than n. A message that converts to an integer of at least n wraps around, so decryption returns m mod n. Large inputs hit this limit; short ones never do.
- **Occasional wrong plaintext:** if one prime is really composite, phi(n) is wrong, so d does not invert e for every m. Some messages still round-trip correctly by chance, which makes the fault look random. Lost leading zeros and float rounding also affect only some inputs.

Debugging Approach

The fault was isolated with a systematic process instead of trial and error.

1. **Reproduce with a minimal case.** Use a tiny key (n = 3233) and a message above n to confirm the wrap-around, and a message that starts with zero bytes to confirm byte loss.
2. **Check key invariants** after generation: p and q are prime, p != q, gcd(e, phi) = 1, (e × d) mod lambda(n) = 1, and n = p × q has the required bit length.
3. **Compare against a trusted reference.** The custom `mod_pow` must equal Python's built-in `pow(m, e, n)` on random inputs; any difference identifies an arithmetic bug.
4. **Run round-trip tests** over edge cases: empty input, single byte, leading zeros, a message one byte shorter than n, a message of exactly n bytes, and multi-block data.
5. **Test primality code on known pseudoprimes** such as 561, 1105 and 1729, which fool a Fermat test.
6. **Fix one fault at a time** and re-run the whole suite, so each fix is shown to remove a specific failure.

Observed results from the reproduction step:

Defect reproduced	**Result observed**	**Meaning**
m >= n (n = 3233, e = 17, d = 2753, m = 4000)	Decryption returns 767	4000 mod 3233 = 767, so the message wrapped around n
Leading zero bytes: b'\x00\x00AB'	Round trip returns b'AB'	Integer conversion lost the leading zeros
Fermat test on 561 (base 2)	pow(2, 560, 561) = 1, so it passes	561 = 3 × 11 × 17 is composite; Miller-Rabin correctly rejects it
Float conversion of a large	float(12345678901234567890) prints	Only about 16 digits survive; the

integer	1.2345678901234567e+19	rest are lost

Part B: Optimization

The optimized design keeps the corrected arithmetic and adds the techniques below.

Technique	Cost	Effect in this system
Square-and-multiply (fast exponentiation)	About $\log_2(e)$ squarings and multiplications instead of e	Makes encryption and decryption feasible at all; with e = 65537 only 17 squarings and 1 multiplication
Chinese Remainder Theorem (CRT)	Two exponentiations with half-size numbers	About 4 times faster private operation in theory (each exponentiation costs about 1/8)
Miller-Rabin with a small-prime sieve	Cheap trial division removes most composites first	Fewer expensive modular exponentiations during prime search
Extended Euclidean algorithm	O(log n)	Replaces brute-force search for d and qinv
Block splitting (or hybrid RSA + AES)	One RSA operation per block, or one per message	Fixes the m >= n failure; hybrid encryption is the practical choice for large data

Fast exponentiation

Square-and-multiply scans the exponent bit by bit. It squares the base at every bit and multiplies by the base when the bit is 1, reducing modulo n at each step. This keeps numbers below n^2 and needs about $\log_2(d)$ steps rather than d steps.

Chinese Remainder Theorem

The private key holds p, q and three precomputed values: dp = d mod (p-1), dq = d mod (q-1) and qinv = q^{-1} mod p. Decryption then becomes:

- m1 = c^dp mod p
- m2 = c^dq mod q
- h = qinv × (m1 - m2) mod p
- m = m2 + h × q

Each exponentiation uses numbers of half the length. With schoolbook multiplication this costs about 1/8 of a full-size exponentiation, and two are needed, so the theoretical gain is about 4 times.

Measured improvement

Key size	Private op, standard (ms)	Private op, CRT (ms)	Speed-up
1024 bits	6.9	2.7	2.6×
2048 bits	39.9	11.2	3.6×
3072 bits	107.2	32.3	3.3×

The measured gain is about 3 to 3.6 times. It is lower than the theoretical 4 times because the CRT timing includes an integrity check, which re-encrypts the result to catch faults. Timings come from the pure-Python code in this document on one machine, so only the ratios matter.

Other fixes

- **Primality:** Miller-Rabin with 40 random bases (error at most 4^{-40}) after a small-prime sieve. Setting the top two bits ensures p × q has the full key length.
- **Large data:** split the message into blocks that always fit below n, or encrypt a random AES key with RSA and the data with AES (hybrid encryption). Hybrid encryption is the standard solution.
- **Padding:** use RSA-OAEP in real systems. The block marker used in the code below only preserves leading zeros and is not a security padding scheme.

Part C: Trade-offs Between Key Size, Security and Performance

Larger keys raise the work needed to factor n, but they also raise the cost of every operation. The table combines NIST SP 800-57 strength estimates with timings measured from the corrected code.

Key size	Security strength (NIST)	Key generation	Decrypt (CRT)	Encrypt (e = 65537)	Ciphertext size	Status
1024	about 80 bits	0.2 s	2.9 ms	0.08 ms	128 bytes	Deprecated
2048	112 bits	1.0 s	10 ms	0.19 ms	256 bytes	Current minimum
3072	128 bits	4.3 s	36 ms	0.43 ms	384 bytes	Recommended for long-term data
4096	about 140 bits	12 s	70 ms	0.90 ms	512 bytes	Higher margin, high cost

Analysis

- **Security:** strength grows slowly with key size. Doubling from 2048 to 4096 bits adds only about 30 bits of estimated strength, because the best factoring attacks (general number field sieve) are sub-exponential.
- **Private operation:** cost grows roughly with the cube of the key size. Measured decryption rose from 10 ms to 70 ms (about 7 times) when the key doubled from 2048 to 4096 bits.
- **Public operation:** with e = 65537, cost grows roughly with the square of the key size (0.19 ms to 0.90 ms, about 4.7 times), so encryption and signature verification stay cheap.

- **Key generation:** cost rises steeply, about 12 times from 2048 to 4096 bits (1.0 s to 12 s), because larger primes are rarer and each test is more expensive. This matters on devices that generate keys often.
- **Storage and bandwidth:** ciphertext and signature size equal the key length (256 bytes at 2048 bits, 512 bytes at 4096 bits).

Further considerations

- **CRT risks:** a fault during CRT computation can leak a prime factor (the Bellcore attack). The code verifies each result by re-encrypting it. Timing attacks call for constant-time arithmetic and blinding in production libraries.
- **Choice of e:** 65537 has only two set bits, which makes encryption fast, and it is large enough to avoid the small-exponent attacks that affect e = 3.
- **Alternatives:** elliptic-curve cryptography gives equivalent strength with much smaller keys (a 256-bit curve is comparable to 3072-bit RSA), so it is often preferred when performance or bandwidth is limited.

Recommendation

Use at least 2048-bit keys with e = 65537 and CRT decryption. Choose 3072 bits for data that must stay confidential beyond about 2030. Move to 4096 bits only when policy demands it, because the extra security margin is small compared with the roughly 7 times slower private operation. Encrypt bulk data with a hybrid scheme and OAEP padding.

Corrected Implementation

The module below is documented and commented. It fixes each defect identified in Part A and includes the optimizations from Part B. It is an educational implementation of textbook RSA; production systems should use a vetted library with OAEP.

```
"""Corrected and optimised textbook-RSA core (educational; use OAEP in production)."""
import secrets
from math import gcd

SMALL_PRIMES = [3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]



def mod_pow(base, exp, mod):
    """Square-and-multiply: O(log exp) multiplications, intermediates stay below mod**2."""
    result, base = 1, base % mod
    while exp > 0:
        if exp & 1:
```

```
            result = (result * base) % mod
        base = (base * base) % mod
        exp >>= 1
    return result



def is_probable_prime(n, rounds=40):
    """Miller-Rabin test with random bases (error probability <= 4**-rounds)."""
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    for sp in SMALL_PRIMES:                    # cheap filter before the costly test
        if n == sp:
            return True
        if n % sp == 0:
            return False
    d, s = n - 1, 0
    while d % 2 == 0:                          # write n-1 = 2^s * d with d odd
        d //= 2
        s += 1
    for _ in range(rounds):
        a = secrets.randbelow(n - 3) + 2       # random base in [2, n-2]
        x = mod_pow(a, d, n)
        if x in (1, n - 1):
            continue
        for _ in range(s - 1):
            x = (x * x) % n
            if x == n - 1:
                break
        else:
            return False                       # definitely composite
    return True
```

```
def generate_prime(bits):
    """Random prime of exactly `bits` bits from a cryptographically secure source."""
    while True:
        # force the top two bits (so p*q has full length) and the low bit (odd)
        cand = secrets.randbits(bits) | (3 << (bits - 2)) | 1
        if is_probable_prime(cand):
            return cand



def mod_inverse(a, m):
    """Modular inverse via the iterative extended Euclidean algorithm."""
    old_r, r, old_s, s = a, m, 1, 0
    while r:
        q = old_r // r
        old_r, r = r, old_r - q * r
        old_s, s = s, old_s - q * s
    if old_r != 1:
        raise ValueError("a has no inverse modulo m")
    return old_s % m                          # normalise a negative result



def generate_keypair(bits=2048, e=65537):
    """Return (public, private); private carries CRT parameters."""
    half = bits // 2
    while True:
        p, q = generate_prime(half), generate_prime(half)
        if p == q or abs(p - q) < (1 << (half - 100)):    # reject equal / close primes
            continue
        lam = (p - 1) * (q - 1) // gcd(p - 1, q - 1)      # Carmichael lambda(n)
        if gcd(e, lam) != 1:                              # e must be invertible
            continue
        d = mod_inverse(e, lam)
        if d.bit_length() < bits // 2:                    # guard against very small d
            continue
        break
    n = p * q
    private = {"n": n, "d": d, "p": p, "q": q,
```

```
            "dp": d % (p - 1), "dq": d % (q - 1), "qinv": mod_inverse(q, p)}
    return (n, e), private


def encrypt_int(m, pub):
    n, e = pub
    if not 0 <= m < n:
        raise ValueError("message integer must satisfy 0 <= m < n")
    return mod_pow(m, e, n)


def decrypt_int(c, priv):
    """CRT decryption (about 4x faster) with a fault check on the result."""
    p, q = priv["p"], priv["q"]
    m1 = mod_pow(c % p, priv["dp"], p)
    m2 = mod_pow(c % q, priv["dq"], q)
    h = (priv["qinv"] * (m1 - m2)) % p
    m = m2 + h * q
    if mod_pow(m, 65537, priv["n"]) != c % priv["n"]:     # detects faulty CRT results
        raise RuntimeError("decryption integrity check failed")
    return m


def encrypt_bytes(data, pub):
    """Split into blocks that always fit below n; a 0x01 marker keeps leading zeros."""
    n, _ = pub
    k = (n.bit_length() + 7) // 8
    size = k - 2                                  # block value < 256**(k-1) < n
    out = []
    for i in range(0, max(len(data), 1), size):
        block = b"\x01" + data[i:i + size]
        c = encrypt_int(int.from_bytes(block, "big"), pub)
        out.append(c.to_bytes(k, "big"))
    return b"".join(out)


def decrypt_bytes(blob, priv):
```

```
    k = (priv["n"].bit_length() + 7) // 8
    out = []
    for i in range(0, len(blob), k):
        m = decrypt_int(int.from_bytes(blob[i:i + k], "big"), priv)
        raw = m.to_bytes((m.bit_length() + 7) // 8, "big")
        out.append(raw[1:])                          # drop the 0x01 marker
    return b"".join(out)
```

Test script:

```
# Round-trip test: includes empty input, leading zeros and multi-block data
import os
pub, priv = generate_keypair(1024)
for msg in [b"", b"hello", b"\x00\x00leading zeros", os.urandom(5000)]:
    assert decrypt_bytes(encrypt_bytes(msg, pub), priv) == msg
print("All round-trip tests passed")
```

Conclusion

The reported failures come from four groups of defects: weak primality testing, incomplete key validation, unsafe exponentiation and arithmetic, and message handling that ignores the requirement $m < n$. Fixing them gives correct round trips. Fast exponentiation and CRT then make the private operation about 3 to 3.6 times faster in measurement. A 2048-bit or 3072-bit key with CRT, hybrid encryption and OAEP gives the best balance of security and performance.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation